МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Реализация численных алгоритмов

Студент
гр. БИН-26-03 _____ К. А. Осмоловский

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1

В первом задании требуется найти наибольший общий делитель двух целых чисел `a` и `b` с помощью алгоритма Евклида. Если хотя бы одно число отрицательное или оба числа равны нулю, необходимо вывести `Ошибка`. Алгоритм решения первого задания представлен на рисунке 1.

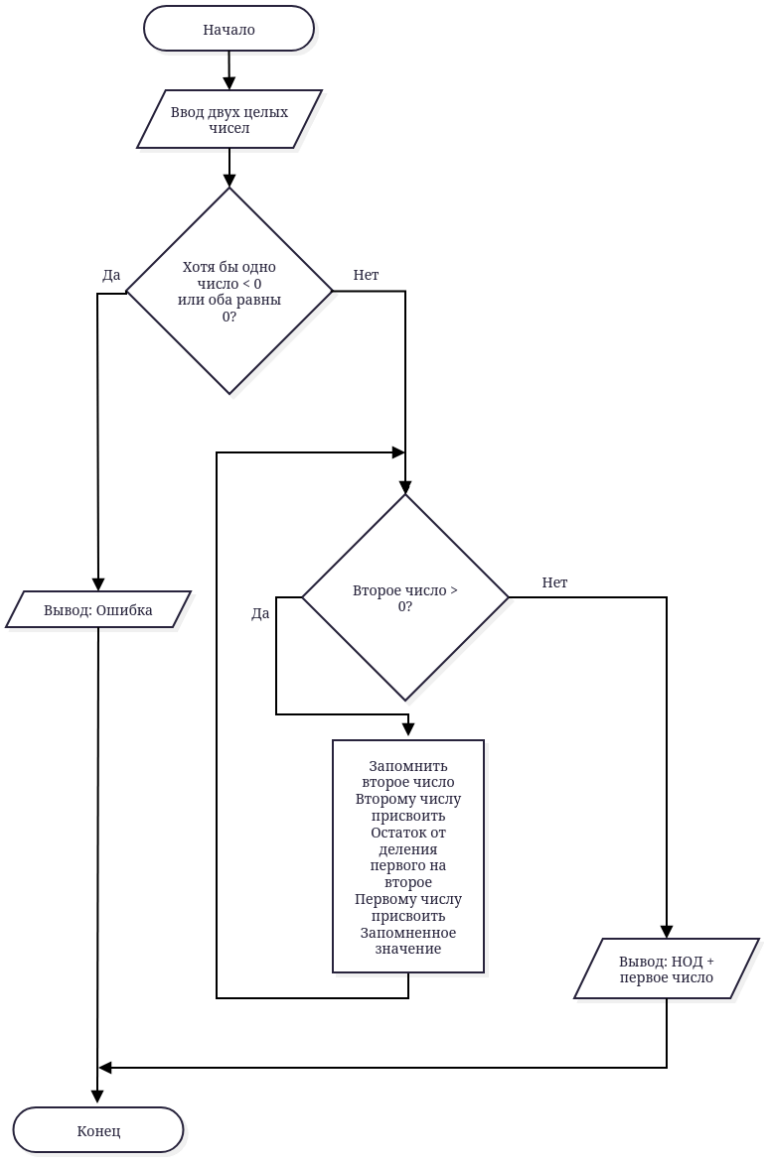


Рисунок 1 – Блок-схема программы для первого задания

Текст программы для первого задания представлен на рисунке 2.

```
a = int(input())
b = int(input())

if a < 0 or b < 0 or ( a == 0 and b == 0):
    print("Ошибка")
else:
    while b > 0:
        c = b
        b = a % b
        a = c
    print(f"НОД: {a}")
```

Рисунок 2 – Текст программы для первого задания

Программа получает на вход две строки и преобразует их к целым числам. Сначала проверяется условие: если хотя бы одно число отрицательное или оба равны нулю, выводится сообщение об ошибке. В противном случае применяется алгоритм Евклида: в цикле while, пока второе число больше нуля, происходит замена пары чисел по правилу НОД(a, b) = НОД(b, a mod b). После завершения цикла первое число содержит результат.

1.2 Задание 2

Во втором задании требуется проверить, является ли введённое целое число `n` простым. Простое число имеет ровно два положительных делителя: 1 и само число. Все значения меньше 2 дают результат `Не простое`. Алгоритм решения второго задания представлен на рисунке 3.

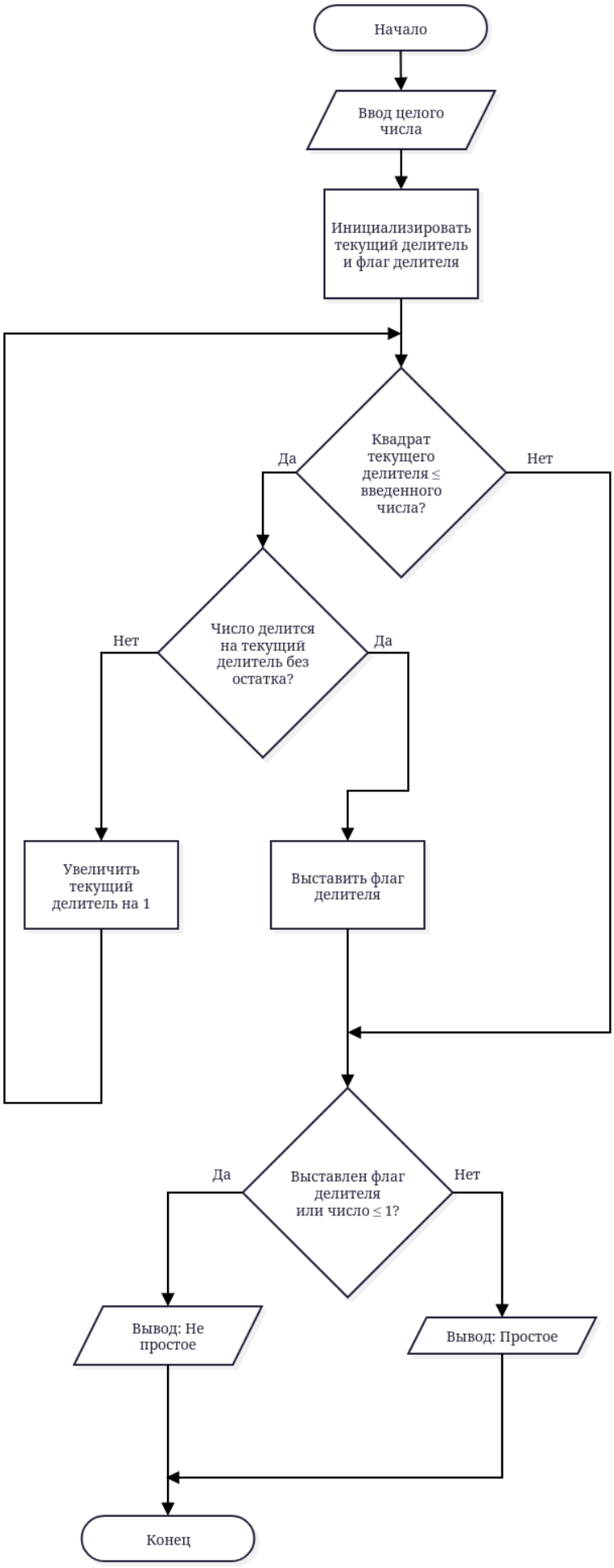


Рисунок 3 – Блок-схема программы для второго задания

Текст программы для второго задания представлен на рисунке 4.

```
n = int(input())
d = 0
i = 2

while i*i <= n:
    if n % i == 0:
        d = i
        break
    i += 1

if d or n <= 1:
    print("Не простое")
else:
    print("Простое")
```

Рисунок 4 – Текст программы для второго задания

Программа получает на вход строку и преобразует её к целому числу. Затем в цикле while перебираются возможные делители начиная с 2, пока квадрат проверяемого делителя не превысит n. Если находится делитель, цикл прерывается. После цикла проверяется: если делитель найден или число меньше либо равно 1, выводится `Не простое`, иначе — `Простое`.

1.3 Задание 3

В третьем задании требуется разложить целое число `n` на простые множители. Если n < 2, выводится `Ошибка`. Множители выводятся в порядке неубывания через пробел. Алгоритм решения третьего задания представлен на рисунке 5.

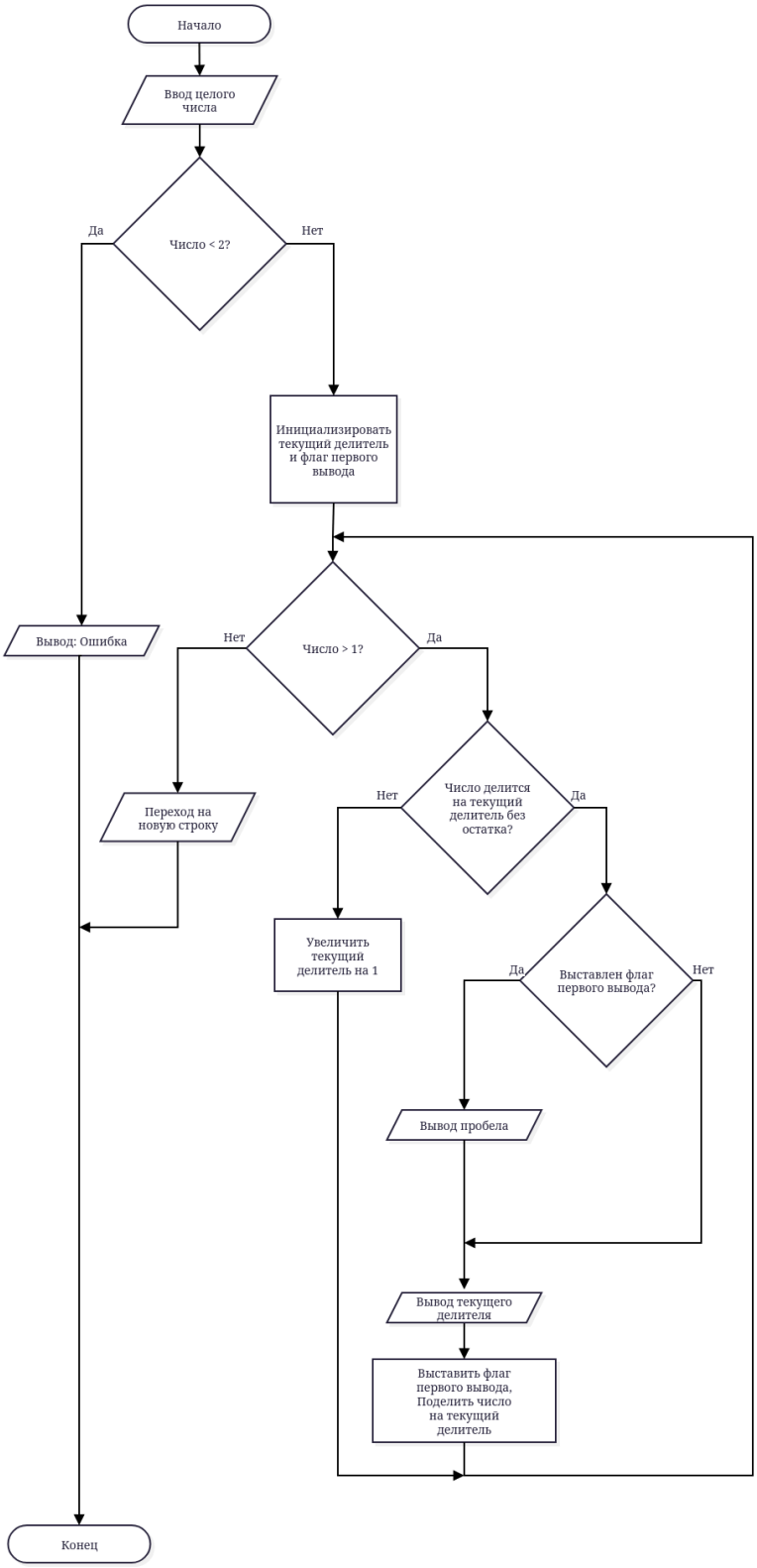


Рисунок 5 – Блок-схема программы для третьего задания

Текст программы для третьего задания представлен на рисунке 6.

```
n = int(input())

if n < 2:
    print("Ошибка")
else:
    i = 2
    first = True
    while n > 1:
        if n % i == 0:
            n //= i
            if not first:
                print(" ", end="")

            print(i, end="")
            first = False
        else:
            i += 1
    print()
```

Рисунок 6 – Текст программы для третьего задания

Программа получает на вход строку и преобразует её к целому числу. Если число меньше 2, выводится сообщение об ошибке. Иначе используется последовательное деление: в цикле while перебираются делители от 2, при делении без остатка число уменьшается, и тот же делитель проверяется повторно. Множители выводятся через пробел с помощью переменной-флага для контроля форматирования.

1.4 Задание 4

В четвёртом задании требуется определить количество цифр и их сумму для введённого целого неотрицательного числа `n`. Если число отрицательное, выводится `Ошибка`. Работа выполняется с числовым значением без преобразования в строку. Алгоритм решения четвёртого задания представлен на рисунке 7.

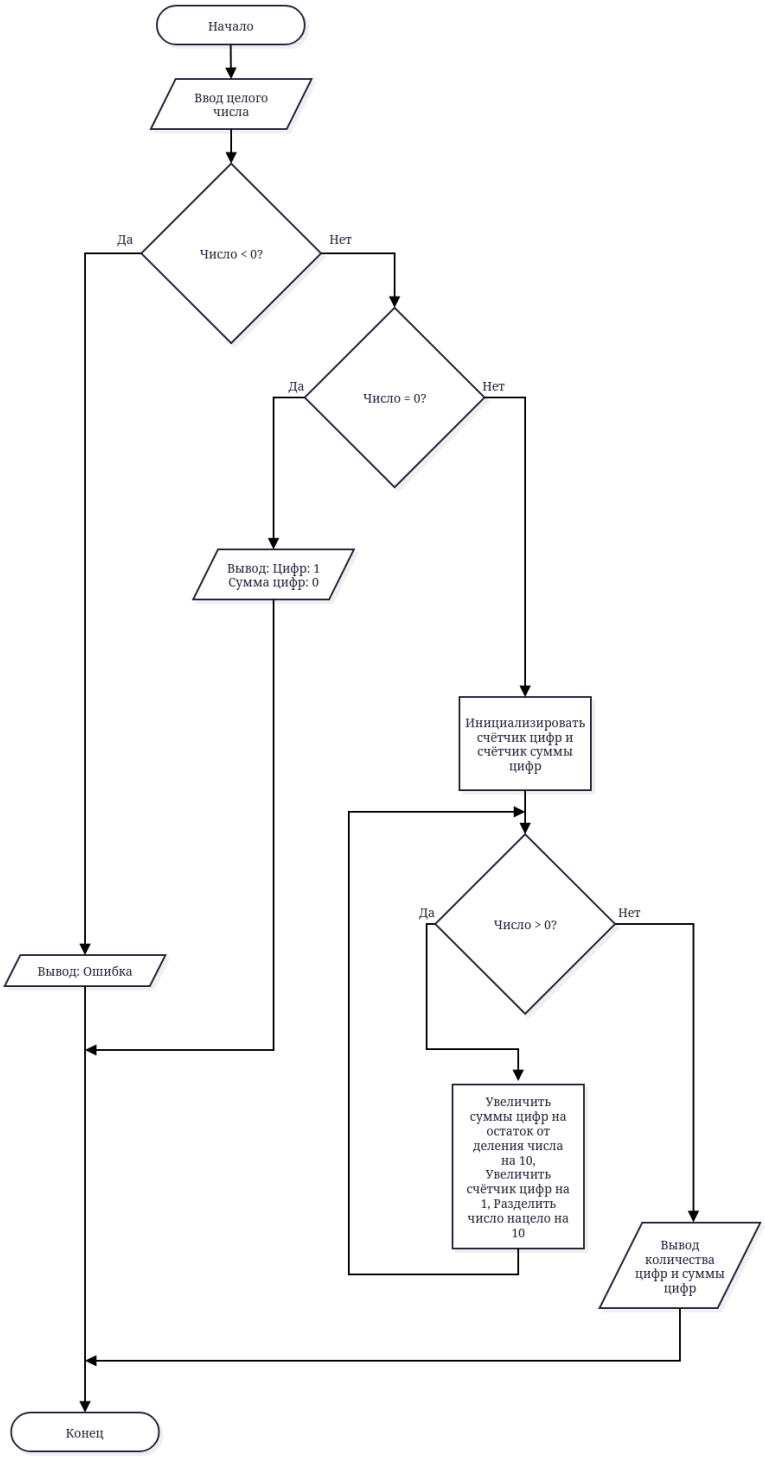


Рисунок 7 – Блок-схема программы для четвёртого задания

Текст программы для четвёртого задания представлен на рисунке 8.

```
n = int(input())

if n < 0:
    print("Ошибка")
elif n == 0:
    print("Цифр: 1\nСумма цифр: 0")
else:
    sum_of_digits = 0
    amount_of_digits = 0
    while n:
        sum_of_digits += n % 10
        amount_of_digits += 1
        n //= 10
    print(f"Цифр: {amount_of_digits}\nСумма цифр: {sum_of_digits}")
```

Рисунок 8 – Текст программы для четвёртого задания

Программа получает на вход строку и преобразует её к целому числу. Если число отрицательное, выводится сообщение об ошибке. Для нуля выводится специальный результат: одна цифра и сумма 0. В остальных случаях в цикле while последовательно выделяется последняя цифра остатком от деления на 10 и добавляется к сумме, а количество цифр увеличивается на 1. Затем число уменьшается целочисленным делением на 10. Цикл продолжается, пока число не станет нулём.

1.5 Задание 5

В пятом задании требуется построить таблицу расстояний между точками с целочисленными координатами от 0 до n-1, где n — целое число от 1 до 10. При выходе за этот диапазон выводится `Ошибка`. Алгоритм решения пятого задания представлен на рисунке 9.

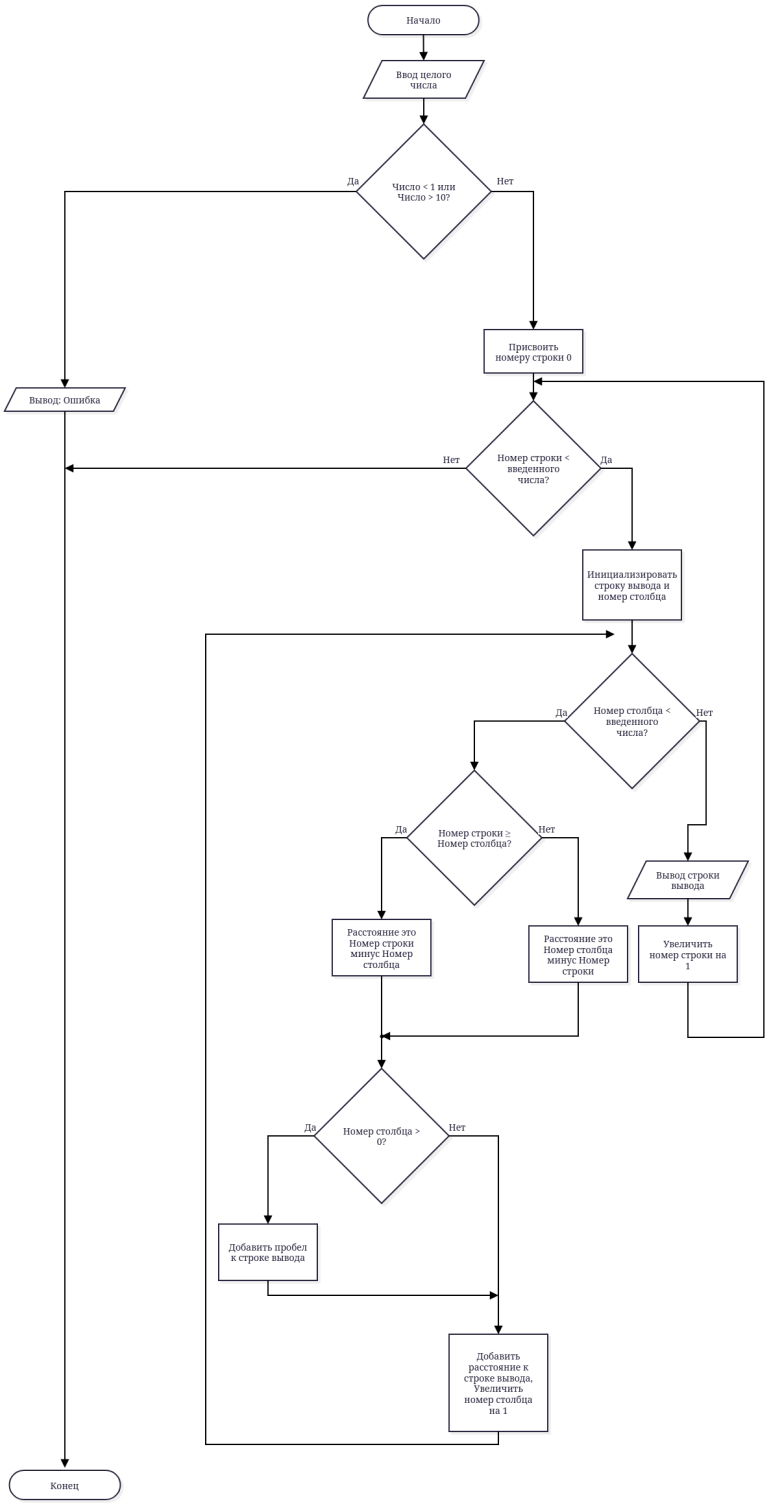


Рисунок 9 – Блок-схема программы для пятого задания

Текст программы для пятого задания представлен на рисунке 10.

```
n = int(input())

if n < 1 or n > 10:
    print("Ошибка")
else:
    for i in range(n):
        s = ""
        for j in range(n):
            if i >= j:
                dist = i - j
            else:
                dist = j - i
            if j > 0:
                s += " "
            s += str(dist)
        print(s)
```

Рисунок 10 – Текст программы для пятого задания

Программа получает на вход строку и преобразует её к целому числу. Если число выходит за диапазон от 1 до 10, выводится сообщение об ошибке. Иначе с помощью вложенных циклов for формируется таблица: внешний цикл перебирает строки, внутренний — столбцы. Для каждой пары координат вычисляется расстояние как модуль разности. Элементы строки разделяются одним пробелом, пробел не добавляется после последнего элемента.